

เอกสารการออกแบบซอฟต์แวร์

Software Design Document (SDD) — ISO/IEC 29110 (SI.3)

โครงการระบบติดตามและแจ้งเตือนพัสดุหอพัก (RMUTL Smart Dormitory Package System)

รายวิชา ENGSE205 Software Process and Quality Assurance

รหัสเอกสาร / Document ID	DOC-SDD-ENGSE205-03 เวอร์ชัน: 2.0.0 (Baseline Software Design)
สถานะการออกแบบซอฟต์แวร์	ผ่านการทบทวนและอนุมัติสถาปัตยกรรม (Approved Software Design Baseline)
ผู้ให้ความต้องการ / ลูกค้า	นางสาวชุนหะกาญจน์ พันธุ์เจริญ (อาจารย์ชุน) — ผู้ว่าจ้างและที่ปรึกษาโครงการ
วันที่อนุมัติแบบสถาปัตยกรรม	17 กันยายน 2569 (เข้าสู่กระบวนการพัฒนาและทดสอบระบบ SI.4 - SI.5)
มาตรฐานอ้างอิง	ISO/IEC 29110 (Basic Profile - SI.3), IEEE Std 1016-2009 (Software Design)
ทีมสถาปนิกและผู้พัฒนา (76 Company)	ณัฐกิตติ รอดเรือน (Lead/SA), ธนโชติ จาติระดุก, มงคล อาษากิจ, วชิรวิทย์ ปิ่นภาไร

บันทึกการแก้ไขและวิวัฒนาการสถาปัตยกรรม (Revision History):

เวอร์ชัน	วันที่	ผู้ปรับปรุง	รายละเอียดการปรับปรุง
0.1.0	3 ก.ย. 2569	ทีม 76 Company	ร่างแบบสถาปัตยกรรมระบบเบื้องต้น (High-Level Architecture Draft)
1.0.0	10 ก.ย. 2569	ทีม 76 Company	ออกแบบ Database Schema, REST Endpoints, และโครงสร้างโมดูลย่อย
2.0.0	17 ก.ย. 2569	ทีม 76 Company	ปรับปรุงสถาปัตยกรรมระบบสมบูรณ์ เพิ่ม BPMN Swimlane, RTM, และ Baseline SI.3

บทที่ 1: บทนำ ขอบเขต และเป้าหมายการออกแบบ (Introduction & Scope)

1.1 วัตถุประสงค์ของเอกสาร (Purpose & ISO/IEC 29110 SI.3 Context)

เอกสารการออกแบบซอฟต์แวร์ (Software Design Document: SDD) ฉบับนี้ จัดทำขึ้นภายใต้กระบวนการวิศวกรรมซอฟต์แวร์ ISO/IEC 29110 (Software Implementation Process: SI.3 Software Architectural and Detailed Design) และมาตรฐาน IEEE Std 1016-2009 เพื่อทำหน้าที่แปลงข้อกำหนดความต้องการที่ผ่านการอนุมัติ (Approved Requirements Baseline: SRS-ENGSE205-2026-V1.0) จากระยะ SI.2 ให้กลายเป็นพิมพ์เขียวทางสถาปัตยกรรม (Architectural Blueprints), แผนผังกระบวนการทำงานเชิงธุรกิจ (BPMN Diagrams), โครงสร้างฐานข้อมูล (Data Models & Schemas), ข้อกำหนดการเชื่อมต่อส่วนต่อประสาน (Interface & API Contracts), และเมทริกซ์การตรวจสอบย้อนกลับ (Requirements Traceability Matrix: RTM) เพื่อเป็นแนวทางที่ชัดเจนและรัดกุมสำหรับทีมผู้พัฒนาในการเขียนโปรแกรม (SI.4 Construction) และการทดสอบระบบ (SI.5 Integration and Testing)

1.2 ขอบเขตระบบและการรองรับข้อกำหนดความต้องการ (Scope & Requirement Alignment)

การออกแบบสถาปัตยกรรมครอบคลุมฟังก์ชันการทำงานหลัก 5 ประการตาม Functional Requirements (FR-01 ถึง FR-05) และ Non-Functional Requirements (NFR-01, NFR-02) ดังนี้:

- REQ-FR01 (การบันทึกและตรวจสอบชื่อ): ออกแบบกลไกตรวจสอบชื่อ Exact Match Verification ผ่าน REST API `/api/students/verify` พร้อม Auto-fill ข้อมูลห้องและ LINE ID
- REQ-FR02 (ระบบแจ้งเตือนส่วนตัว): ออกแบบบริการ Notification Service ผสานการทำงานกับ mock-line-notify.js ส่งแจ้งเตือนจำลองระดับมิลลิวินาที
- REQ-FR03 (การแจ้งเตือนพัสดุค้างรับ): ออกแบบกลไก Background / Scheduled Scanner ผ่าน `/api/notifications/check-reminders` ตรวจสอบพัสดุ pending เกิน 5 ชม.
- REQ-FR04 (บอร์ดประกาศพัสดุไม่ทราบชื่อ): ออกแบบระบบ Broadcast Board และ Workflow การยื่นขอเคลมพัสดุ (`POST /claim`) สำหรับนักศึกษา
- REQ-FR05 (การเซ็นรับดิจิทัล): ออกแบบพื้นผ้าใบ Canvas สำหรับลงลายมือชื่อบนหน้าจอนักศึกษา แปลงเป็น Base64 Data URL บันทึกพร้อมประทับเวลา pickup_time
- REQ-NFR (คุณภาพและประสิทธิภาพ): ออกแบบสถาปัตยกรรมให้ API มี Latency เฉลี่ยต่ำกว่า 200 ms, มี Centralized Error Handling, และจัดเก็บข้อมูลอย่างคงทนบน MongoDB

บทที่ 2: สถาปัตยกรรมระบบและโครงสร้างเลเยอร์ (System Architecture)

ระบบถูกออกแบบตามรูปแบบสถาปัตยกรรม Multi-tier Client-Server with Layered Service Architecture โดยแบ่งออกเป็น 3 ระดับชั้น (Tiers) หลักอย่างชัดเจน เพื่อความยืดหยุ่นในการขยายระบบ (Scalability), ความปลอดภัย (Security), และความง่ายในการบำรุงรักษา (Maintainability):

1. Client Tier (ระดับชั้นลูกค้าผู้ใช้งาน)

ประกอบด้วย 2 พอร์ทัลเว็บแอปพลิเคชันที่พัฒนาด้วย React 19 + Vite:

- Staff Portal (พอร์ทัล 5174): สำหรับเจ้าหน้าที่หอพัก มี Dashboard ภาพรวม, ตารางจัดการพัสดุ,

แบบฟอร์มบันทึกพร้อมระบบตรวจสอบชื่ออัตโนมัติ, และโมดูลจัดการพัสดุไม่ทราบชื่อ

- Student Portal (พอร์ทัล 5173): สำหรับนักศึกษา ออกแบบรองรับ Responsive Mobile-First เพื่อให้นักศึกษาเปิดบนสมาร์ตโฟนได้

มีกระดานประกาศพัสดุไม่ทราบชื่อ (Broadcast Board) และหน้าจอลงนามเซ็นรับพัสดุดิจิทัล (Digital Signature Canvas)

- Unified API Client Layer (`apiClient.js`): ทำหน้าที่เป็นสะพานเชื่อมต่อ API ส่วนกลาง จัดการ Base URL ตาม Environment Variables, จัดการ Header JSON, และทำ Error Normalization อย่างเป็นมาตรฐาน

2. Server Tier (ระดับชั้นเซิร์ฟเวอร์ประมวลผล)

พัฒนาด้วย Node.js และ Express.js ทำงานบนพอร์ต 5000 โดยแบ่งเป็นเลเยอร์ย่อย:

- Central Configuration & Middleware: จัดการ CORS Whitelist, Body Parser ขนาด 10 MB, และ Morgan HTTP Request Logger
- Routing & Controller Layer: แยกความรับผิดชอบตามทรัพยากร (Packages, Students, Notifications) และห่อหุ้มด้วย `asyncHandler`
- Business Logic & Service Layer: จัดการตรวจสอบ Exact Match, ควบคุมสถานะพัสดุ, ตรวจสอบพัสดุเกิน 5 ชม., และผสานกับโมดูลจำลองการส่งแจ้งเตือน (`mockSendNotification`)
- Centralized Error Handler: ดักจับ Error ทั้งระบบและแปลงเป็น Standard JSON Response

3. Database Tier (ระดับชั้นฐานข้อมูล)

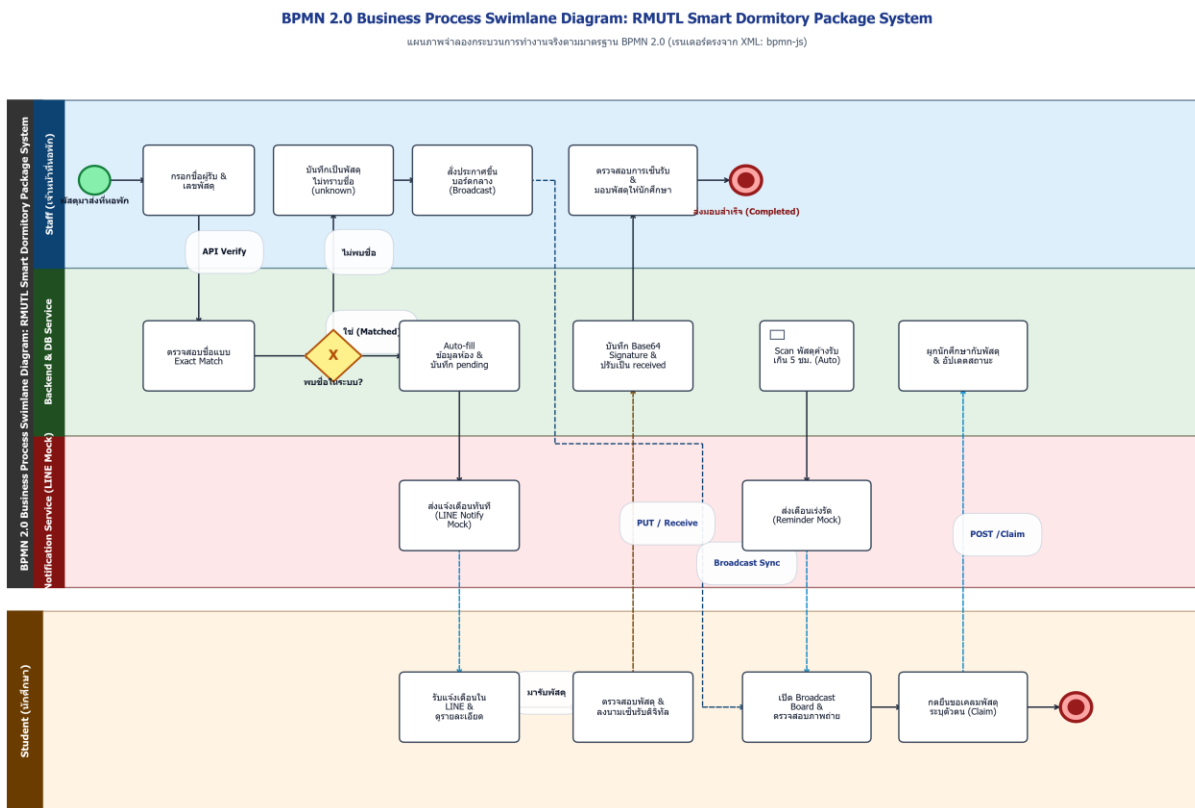
ใช้งาน MongoDB ทำงานบนพอร์ต 27017 โดยเข้าถึงผ่าน Mongoose ODM (Object Data Modeling) จัดเก็บข้อมูล 3 คอลเลกชันหลัก ได้แก่ `students`, `packages`, และ `notifications` พร้อมการทำดัชนี (Indexing) บนฟิลด์ที่สืบค้นบ่อย

บทที่ 3: แผนภาพกระบวนการทำงาน BPMN (Business Process Model & Notation)

ตามข้อกำหนดของมาตรฐาน ISO/IEC 29110 ในขั้นตอน SI.3 การออกแบบต้องแสดงกระบวนการทำงานทั้งในระดับธุรกิจ (Business Process) และในระดับวงจรชีวิตการพัฒนา (Software Process Lifecycle) อย่างเป็นรูปธรรม:

3.1 แผนภาพ BPMN กระบวนการทำงานทางธุรกิจ (Business Process Swimlane Diagram)

แผนภาพว่าด้วยการทำงานข้ามเลน (Swimlane) 4 บทบาท: เจ้าหน้าที่หอพัก (Staff), ระบบหลังบ้านและฐานข้อมูล (Backend & DB), ระบบจำลองการแจ้งเตือน (Notification Service), และนักศึกษา (Student) ครอบคลุมวงจรชีวิตพัสดุครบทั้ง 3 โฟลว์หลัก:



รูปที่ 3.1: แผนภาพ BPMN 2.0 Swimlane แสดงกระบวนการจัดการพัสดุหอพักแบบครบวงจร

ตารางรายละเอียดขั้นตอนการทำงานตาม Swimlane (BPMN Detailed Activities):

ขั้นตอน	บทบาท (Lane)	กิจกรรม / เหตุการณ์ (Task/Event)	ข้อมูลเข้า/ออก (I/O)	ข้อกำหนด
Step 1	Staff (เจ้าหน้าที่)	พัสดุมาถึง -> กรอกข้อมูลผู้รับและเลขพัสดุลงในแบบฟอร์ม	ชื่อ, เลขพัสดุ, รูปถ่าย	FR-01
Step 2	Backend & DB	ตรวจสอบชื่อ Exact Match เทียบกับคอลเลกชัน students	Query: receiver_name -> Matched Student	FR-01
Step 3A	Backend & Staff	[กรณีพบชื่อ] Auto-fill ข้อมูลห้อง/LINE - > บันทึกสถานะ pending	Status: 'pending', matched_student: ID	FR-01
Step 3B	Backend & Staff	[กรณีไม่พบชื่อ] จัดเข้าหมวด unknown - > ส่ง Broadcast ขึ้นบอร์ดกลาง	Status: 'unknown', is_broadcasted: true	FR-01, FR-04
Step 4	Notification Service	ส่ง Mock LINE Notify แจ้งเตือนไปยัง LINE ของนักศึกษาทันที	Payload: ชื่อ, ห้อง, เลขพัสดุ, เวลา	FR-02
Step 5	Student (นักศึกษา)	ตรวจสอบบอร์ด/รับแจ้งเตือน -> ยื่นเคลม (ถ้าเป็น unknown) -> มารับของ	Student ID, Claim Form	FR-04, FR-05

3.2 แผนภาพความสัมพันธ์และวงจรชีวิตตามมาตรฐาน ISO/IEC 29110 (VSE Process Lifecycle)

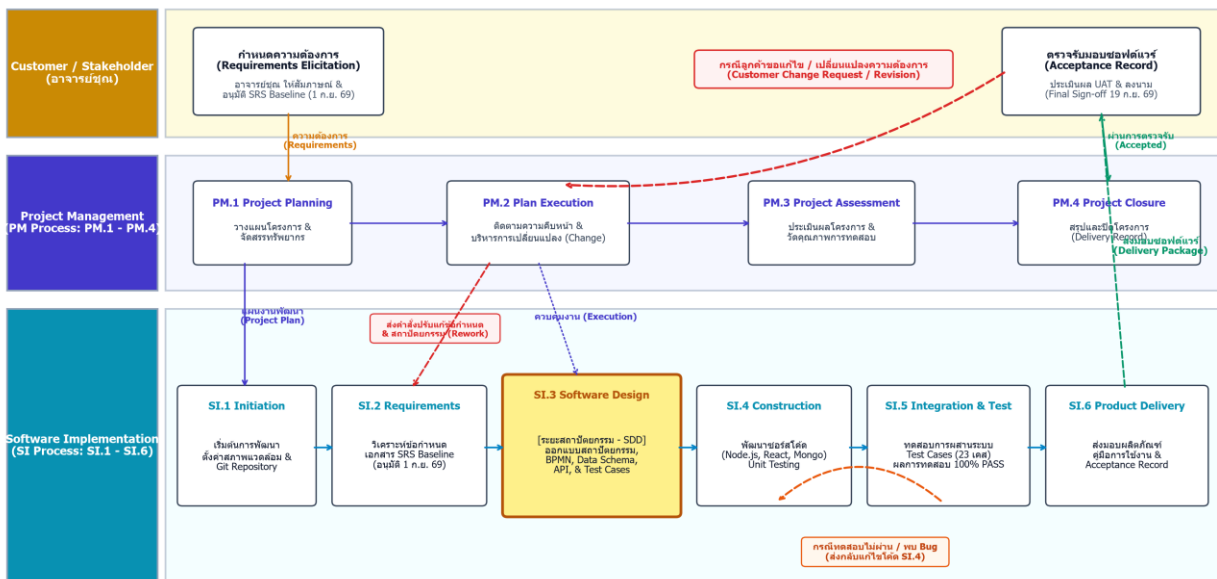
กระบวนการพัฒนาซอฟต์แวร์ของโครงการถูกกำกับดูแลตามกรอบมาตรฐาน ISO/IEC 29110 (Basic Profile)

โดยแบ่งความสัมพันธ์ออกเป็น 3 เลเยอร์หลัก: ผู้มีส่วนได้ส่วนเสีย/ลูกค้า (Customer), กระบวนการบริหารโครงการ (Project Management: PM.1 ถึง PM.4), และกระบวนการดำเนินการพัฒนาซอฟต์แวร์ (Software Implementation: SI.1 ถึง SI.6)

ISO/IEC 29110 VSE Basic Profile: Software Implementation & Management Process Lifecycle

กระบวนการพัฒนาซอฟต์แวร์ตามมาตรฐาน ISO/IEC 29110:2018 (VSE Basic Profile) สำหรับโครงการพัฒนาซอฟต์แวร์ (Software Development Project)

พร้อมเส้นทางการสะท้อนกลับเพื่อแก้ไขงาน (Feedback & Change Request Loops)



รูปที่ 3.2: แผนภาพกระบวนการและวงจรชีวิตตามมาตรฐาน ISO/IEC 29110 (Customer, PM, and SI Processes)

บทที่ 4: การออกแบบระบบย่อยและคอมโพเนนต์ (Subsystem & Component Design)

โครงสร้างโปรแกรมถูกแยกออกเป็น 4 ระบบย่อย (Subsystems) เพื่อให้สอดคล้องกับหลักการ Single Responsibility และ Separation of Concerns:

4.1 Staff Portal Subsystem (`frontend/staff` - Port 5174)

รับผิดชอบส่วนติดต่อผู้ใช้งานสำหรับเจ้าหน้าที่หอพัก ประกอบด้วยคอมโพเนนต์หลัก:

- `App.jsx`: State Container หลัก จัดการ State ของพัสดุ (packages), นักศึกษา (students), และการกรองข้อมูล
- `PackageForm.jsx`: แบบฟอร์มบันทึกพัสดุ เชื่อมต่อกับ `/api/students/verify` เพื่อตรวจสอบชื่อ Exact Match แบบ Real-time ทันทีที่พิมพ์ชื่อครบ
- `PackageManagement.jsx`: ตารางแสดงรายการพัสดุพร้อมตัวกรองสถานะ (All, Pending, Received, Unknown) และปุ่มปรับสถานะ
- `UnknownPackages.jsx`: การแจ้งเตือนรายการพัสดุไม่ทราบชื่อ พร้อมปุ่ม Broadcast เพื่อส่งกระจายพัสดุขึ้นบอร์ดกลาง
- `StatCard.jsx`: แสดงสถิติจำนวนพัสดุทั้งหมด, รอมารับ, รับแล้ว, และไม่ทราบชื่อ
- `ConfirmDeleteModal.jsx`: ป๊อปอัพยืนยันความปลอดภัยก่อนลบประวัติพัสดุ

4.2 Student Portal Subsystem (`frontend/student` - Port 5173)

รับผิดชอบส่วนติดต่อผู้ใช้งานสำหรับนักศึกษา ออกแบบรองรับการใช้งานบนอุปกรณ์มือถือ (Mobile-Responsive):

- `App.jsx`: จัดการการค้นหาพัสดุของตนเองผ่านชื่อ-นามสกุล หรือรหัสนักศึกษา
- `BroadcastBoard.jsx`: กระดานแสดงพัสดุไม่ทราบชื่อที่ถูก Broadcast มาจากเจ้าหน้าที่ มีระบบยืนยันขอเคลม (Claim Package)
- `SignatureCanvas`: หน้าต่างพื้นผิวแบบ HTML5 Canvas ให้นักศึกษาใช้นิ้วหรือปากกาวาดลายมือชื่อดิจิทัล และส่งออกเป็น Base64 Data URL

4.3 Backend API Subsystem (`backend/src` - Port 5000)

รับผิดชอบการประมวลผล Business Logic ทั้งหมด:

- `packageController.js`: ควบคุมการทำงาน CRUD พัก, การเปลี่ยนสถานะ, การส่ง Broadcast, การเคลม, และการยืนยันการรับพร้อมลายเซ็น
- `studentController.js`: ควบคุมการตรวจสอบชื่อ Exact Match และการดึงข้อมูลทะเบียนนักศึกษา
- `notificationService.js`: รับคำสั่งแจ้งเตือนและผลการทำงานกับโมดูลจำลองภายนอก
- `errorHandler.js`: จัดการข้อผิดพลาดส่วนกลาง คืนผลลัพธ์เป็นมาตรฐาน JSON พร้อมแปลง Status Code

4.4 Notification Mockup Service (`notify-mockup` Subsystem)

โมดูลจำลองการทำงานของ LINE Notify (`mock-line-notify.js`):

- จำลองการส่ง HTTP POST ไปยัง LINE Notify API Gateway
- จัดรูปแบบข้อความแจ้งเตือนด้วยไอคอนและโครงสร้างข้อความสวยงาม
- แสดงผลลัพธ์การส่งผ่าน Server Console พร้อมบันทึก Timestamp และสถานะการส่ง (Success Rate: 100%)

บทที่ 5: โครงสร้างฐานข้อมูลและพจนานุกรมข้อมูล (Database Schema Design)

ระบบใช้ฐานข้อมูลแบบ NoSQL Document-Oriented (MongoDB) บริหารจัดการผ่าน Mongoose ODM มี 3 คอลเลกชันหลัก:

5.1 คอลเลกชัน: `students` (ทะเบียนข้อมูลนักศึกษาหอพัก)

ชื่อฟิลด์ (Field)	ชนิดข้อมูล	ข้อกำหนด (Constraints)	คำอธิบาย
_id	ObjectId	Primary Key (Auto-generated)	รหัสระบุเป็นเอกสารเฉพาะใน MongoDB
student_id	String	Required, Unique, Trimmed	รหัสนักศึกษา (เช่น '651234567-8')
first_name	String	Required, Trimmed	ชื่อจริงภาษาไทยของนักศึกษา
last_name	String	Required, Trimmed	นามสกุลภาษาไทยของนักศึกษา
building	String	Required	อาคารหอพัก (เช่น 'หอพักชาย 1', 'S20')
room_number	String	Required	หมายเลขห้องพัก (เช่น '202', '314')
line_user_id	String	Optional (Indexed)	LINE User ID หรือ LINE Account Handle

5.2 คอลเลกชัน: `packages` (ข้อมูลพัสดุและวงจรสถานะ)

ชื่อฟิลด์ (Field)	ชนิดข้อมูล	ข้อกำหนด (Constraints)	คำอธิบาย
_id	ObjectId	Primary Key (Auto-generated)	รหัสระบุเป็นพัสดุเฉพาะใน MongoDB
tracking_number	String	Required, Indexed	เลขพัสดุนกกล่อง (Tracking Number)
receiver_name_on_box	String	Required, Trimmed	ชื่อผู้รับที่จำหน่ายบนกล่องพัสดุ
matched_student	ObjectId	Ref: 'Student', Nullable	การเชื่อมโยง Foreign Key ไปยัง Student
status	String	Enum: ['pending', 'received', 'unknown']	สถานะพัสดุ (Default: 'pending')
is_broadcasted	Boolean	Default: false	สถานะการประกาศขึ้นบอร์ดกลาง (FR-04)
image_path	String	Optional (Base64/URL)	รูปถ่ายกล่องพัสดุลำหรับช่วยระบุตัวตน
arrival_time	Date	Default: Date.now	วันและเวลาที่พัสดุถูกบันทึกเข้าระบบ
pickup_time	Date	Default: null	วันและเวลาที่นักศึกษามารับพัสดุจริง
signature_data	String	Long String (Base64 Data URL)	ข้อมูลภาพลายเซ็นดิจิทัลของนักศึกษา (FR-05)
reminded_at	Date	Default: null	วันเวลาที่ส่งแจ้งเตือนครั้งสุดท้าย (FR-03)

5.3 คอลเลกชัน: `notifications` (ประวัติการส่งแจ้งเตือน)

ชื่อฟิลด์ (Field)	ชนิดข้อมูล	ข้อกำหนด (Constraints)	คำอธิบาย
_id	ObjectId	Primary Key (Auto-generated)	รหัสระบุเป็นประวัติการแจ้งเตือน
recipient	String	Required	ผู้รับการแจ้งเตือน (ชื่อนักศึกษา หรือ LINE ID)
package_id	ObjectId	Ref: 'Package', Required	รหัสพัสดุที่เกี่ยวข้องกับการแจ้งเตือน

type	String	Enum: ['arrival', 'reminder_5h', 'broadcast']	ประเภทการแจ้งเตือน
message	String	Required	ข้อความแจ้งเตือนฉบับสมบูรณ์ที่ถูกจัดรูปแบบ
sent_at	Date	Default: Date.now	วันที่ระบบทำการส่งข้อความสำเร็จ

บทที่ 6: สถาปัตยกรรมส่วนต่อประสานและสัญญา API (Interface & API Contracts)

ระบบสื่อสารระหว่าง Client Tier และ Server Tier ผ่าน RESTful JSON API รวมทั้งสิ้น 16 Endpoints ตามที่ระบุไว้ใน

`API\_CONTRACT.md`:

หมวดหมู่	Method	Endpoint URI	คำอธิบาย / รหัสความต้องการ
System	GET	/	Health Check ตรวจสอบสถานะเซิร์ฟเวอร์
Verification	GET	/api/students/verify	ตรวจสอบชื่อ Exact Match & Auto-fill (FR-01)
Students	GET	/api/students	ดึงรายชื่อนักศึกษาทั้งหมดในระบบ
Students	POST	/api/students	เพิ่มข้อมูลนักศึกษาใหม่เข้าระบบ
Packages	GET	/api/packages	ดึงรายการพัสดุทั้งหมด (รองรับ status filter)
Packages	POST	/api/packages	บันทึกพัสดุใหม่ พร้อมส่ง LINE Notify (FR-01, FR-02)
Packages	GET	/api/packages/:id	ดึงข้อมูลพัสดुरายชิ้นตาม Object ID
Packages	PUT	/api/packages/:id	แก้ไขข้อมูลพัสดุ (Tracking, Receiver, Status)
Packages	DELETE	/api/packages/:id	ลบข้อมูลพัสดุออกจากรายชื่อข้อมูล
Packages	PUT	/api/packages/:id/status	เปลี่ยนสถานะพัสดุ (pending, received, unknown)
Packages	PUT	/api/packages/:id/receive	ยืนยันการรับพัสดุพร้อมบันทึก Digital Signature (FR-05)
Broadcast	POST	/api/packages/broadcast	ส่งประกาศพัสดุไม่ทราบชื่อขึ้นบอร์ดกลาง (FR-04)
Broadcast	GET	/api/packages/broadcasts	ดึงรายการพัสดุที่กำลังประกาศบนบอร์ดกลาง (FR-04)
Broadcast	POST	/api/packages/:id/claim	นักศึกษายืนยันขอเคลมพัสดุว่าเป็นของตนเอง (FR-04)
Notification	GET	/api/notifications/check-reminders	สแกนพัสดุดังรับเกิน 5 ชม. และส่งเตือนซ้ำ (FR-03)
Maintenance	POST	/api/packages/reset-database	รีเซ็ตฐานข้อมูลเป็นค่าเริ่มต้นสำหรับการทดสอบ

บทที่ 7: สถาปัตยกรรมการจัดการข้อผิดพลาดและบันทึกระบบ (Error Handling & Logging)

เพื่อความเสถียรและความน่าเชื่อถือของระบบ ระบบได้ออกแบบ Centralized Error Handling Pipeline ตามหลัก Clean Architecture:

7.1 โครงสร้างการดักจับข้อผิดพลาด (Centralized Error Pipeline)

ทุก Controller จะถูกห่อหุ้มด้วย `asyncHandler` เพื่อดักจับ Async Rejection อัดโนมิตี และเมื่อเกิด Business Logic Error จะทำการโยน `AppError(message, statusCode)` ส่งต่อไปยัง `errorHandler` ซึ่งเป็น Middleware ตัวสุดท้ายของ Express:

```
// ตัวอย่าง Standard Error JSON Response
{
  "success": false,
  "status": "fail",
  "message": "ไม่พบพัสดุในระบบ",
  "statusCode": 404
}
```

7.2 การบันทึกการทำงานของระบบ (HTTP Request Logging with Morgan)

ระบบติดตั้ง Morgan Logger ที่ `backend/src/app.js` ในรูปแบบ `:method :url :status :response-time ms - :res[content-length]` เพื่อบันทึกทุกคำขอที่เข้ามายัง API เซิร์ฟเวอร์แบบเรียลไทม์ สนับสนุนการตรวจสอบย้อนกลับ (Audit Trail) และการวิเคราะห์ปัญหาในกระบวนการทดสอบ

บทที่ 8: สถาปัตยกรรมความปลอดภัยและคุณลักษณะเชิงคุณภาพ (Security & NFRs)

- CORS Whitelist Protection:** กำหนด Whitelist เฉพาะ Client Origin ที่ได้รับอนุญาต (http://localhost:5173, http://localhost:5174, http://localhost:3000) เพื่อป้องกันการโจมตีแบบ Cross-Origin Unauthorized Access
- Payload Size Limit (10MB):** กำหนดข้อจำกัดขนาดของ Request Body สูงสุดที่ 10 MB (`express.json({ limit: '10mb' })`) เพื่อรองรับการส่งภาพถ่ายพัสดุและภาพถ่ายลายมือชื่อดิจิทัล Base64 Data URL โดยป้องกันปัญหา Denial of Service จาก Payload ขนาดใหญ่เกินควร
- Data Persistence & Integrity:** มีกลไก Mongoose Schema Validation ตรวจสอบความถูกต้องของข้อมูลก่อนบันทึกลงใน MongoDB และจัดการ Connection Pool อย่างมีประสิทธิภาพ
- Responsive & Mobile UX Design:** Student Portal ถูกออกแบบด้วย Mobile-First Layout เพื่อให้นักศึกษาสามารถเปิดผ่านสมาร์ทโฟนเพื่อลงลายมือชื่อดิจิทัลผ่านหน้าจอได้สะดวก

บทที่ 9: เมทริกซ์การตรวจสอบย้อนกลับการออกแบบ (Design Traceability Matrix)

ตามมาตรฐาน ISO/IEC 29110 (SI.3) เอกสารการออกแบบต้องแสดงความเชื่อมโยงย้อนกลับ (Traceability) ครบ 100% ระหว่างข้อกำหนดความต้องการ (SRS) -> คอมโพเนนต์สถาปัตยกรรม -> โครงสร้างข้อมูล -> API Endpoint -> ชุดการทดสอบ (Test Cases):

รหัสข้อกำหนด (SRS)	คอมโพเนนต์การออกแบบ	คอลเลกชันฐานข้อมูล	API Endpoints	ชุดการทดสอบ (Test Cases)
REQ-FR01	PackageForm, studentController	students, packages	/api/students/verify, /api/packages	TC-FR01-01 ถึง 05
REQ-FR02	notificationService, mock-line	notifications	POST /api/packages	TC-FR02-01 ถึง 03
REQ-FR03	notificationService, Scanner	packages, notifications	GET /api/notifications/check-reminders	TC-FR03-01 ถึง 03
REQ-FR04	UnknownPackages, BroadcastBoard	packages	/api/packages/broadcast, /claim	TC-FR04-01 ถึง 04
REQ-FR05	SignatureCanvas, packageController	packages (signature_data)	PUT /api/packages/:id/receive	TC-FR05-01 ถึง 04
REQ-NFR01	Central Express API, Morgan	ทุกคอลเลกชัน	ทุก Endpoints (Latency < 200ms)	TC-NFR-01
REQ-NFR02	MongoDB, Mongoose ODM	students, packages, notifs	Data Persistence across restarts	TC-NFR-02

บทที่ 10: การลงนามอนุมัติแบบสถาปัตยกรรม (Software Design Baseline Sign-Off)

ตามกระบวนการ ISO/IEC 29110 (SI.3.5 Establish Baseline)

สถาปัตยกรรมและรายละเอียดการออกแบบทั้งหมดในเอกสารฉบับนี้

ได้รับการพิจารณาทบทวนและลงนามอนุมัติร่วมกันโดยทีมสถาปนิกและผู้พัฒนาซอฟต์แวร์

เพื่อใช้เป็นพิมพ์เขียวหลักในการเขียนโปรแกรม (SI.4) และการทดสอบระบบ (SI.5):

ลำดับ	ชื่อ-นามสกุล ผู้รับผิดชอบ	บทบาทหน้าที่ในการออกแบบ	ลายมือชื่อ / วันที่
1	นายณัฐกิตติ์ รอดเรือน	Project Lead & System Architect (ผู้ออกแบบสถาปัตยกรรมรวมและ BPMN)	ณัฐกิตติ์ รอดเรือน (17 ก.ย. 2569)
2	นายธนโชติ จาติระดุก	Staff Subsystem Lead (ผู้ออกแบบส่วนพอร์ทัลเจ้าหน้าที่)	ธนโชติ จาติระดุก (17 ก.ย. 2569)
3	นายมงคล อาษากิจ	Backend & Service Lead (ผู้ออกแบบระบบหลังบ้าน & ฐานข้อมูล)	มงคล อาษากิจ (17 ก.ย. 2569)
4	นายวชิรวิทย์ ปินะกาโร	Student Subsystem Lead (ผู้ออกแบบส่วนพอร์ทัลนักศึกษา)	วชิรวิทย์ ปินะกาโร (17 ก.ย. 2569)